Lab

2

TRUYỀN TẢI THÔNG TIN

Môn học: Truyền dữ liệu

GVHD: Trần Văn Như Ý

Tháng 08/2026

Lưu hành nội bộ

Sinh viên thực hiện	Lê Hoàn Phúc
Thời gian thực hiện	14/10/2026 – 20/10/2026

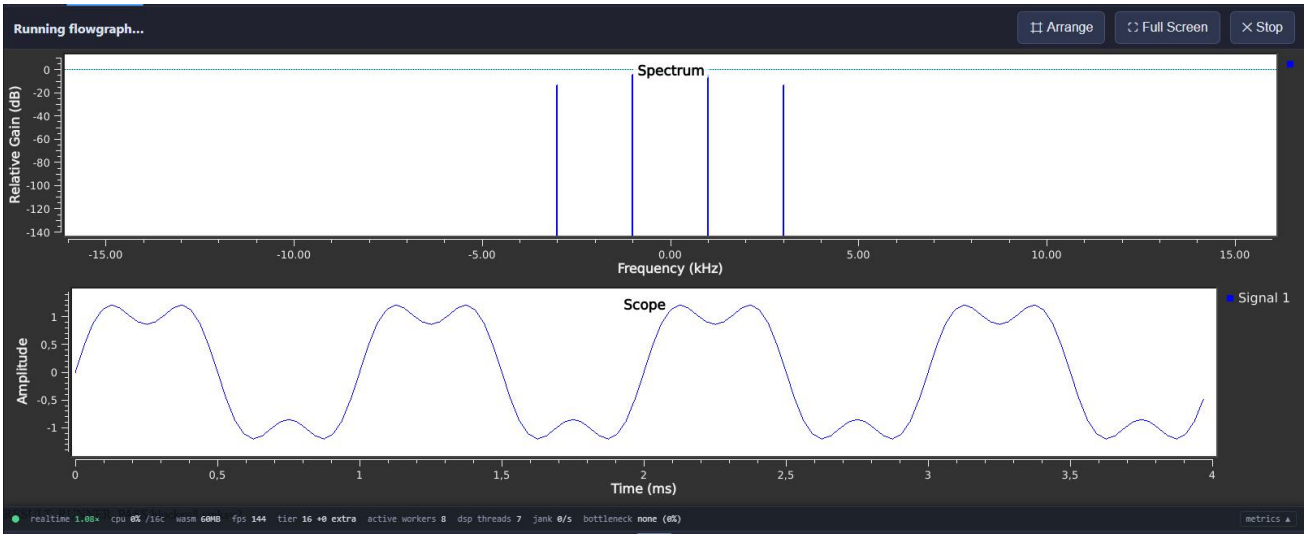
B. Bài tập với GNU Radio Companion

7) Dựng lại flowgraph như Hình 2-8 và thực hiện:

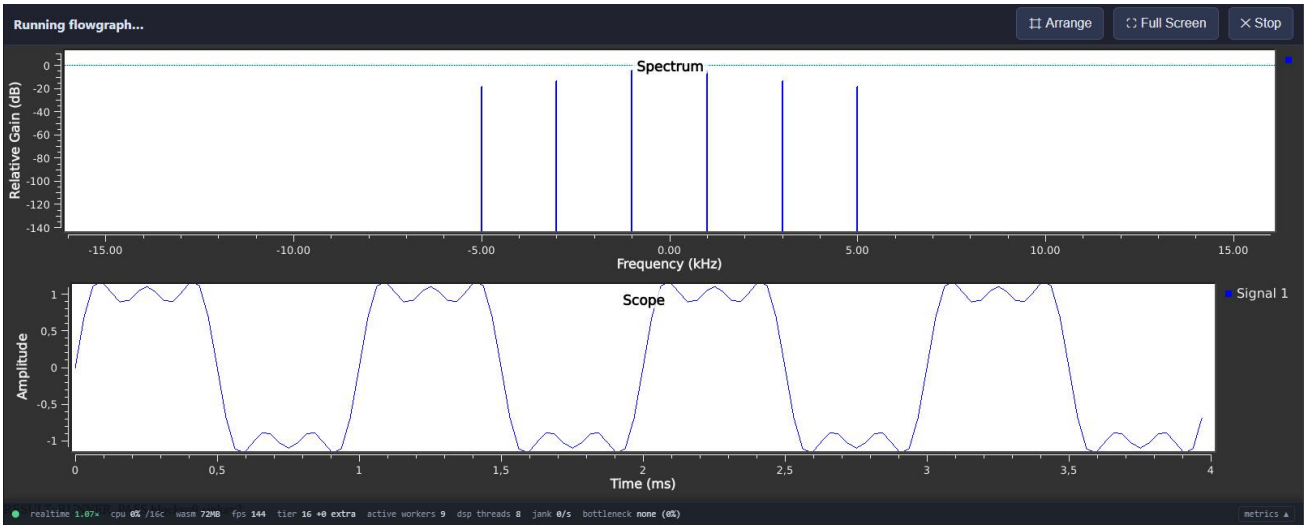
- Chạy với k = 3 (hai nguồn) rồi k = 5 (ba nguồn). Chụp lại cả Time Sink và Frequency Sink trong hai trường hợp.

Trường hợp:

- k = 3:



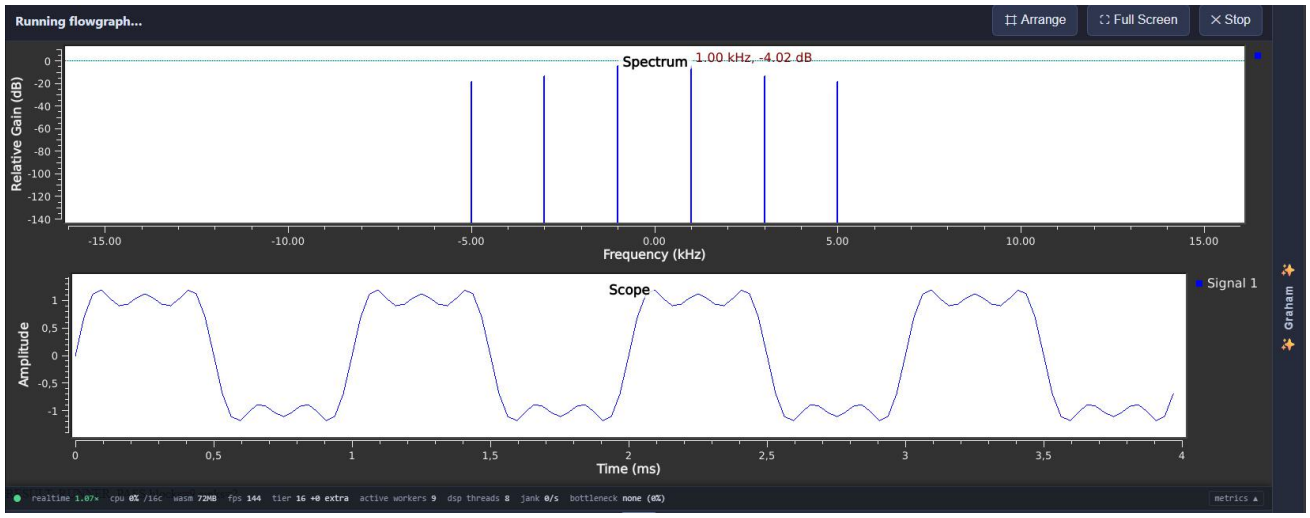
- k = 5:



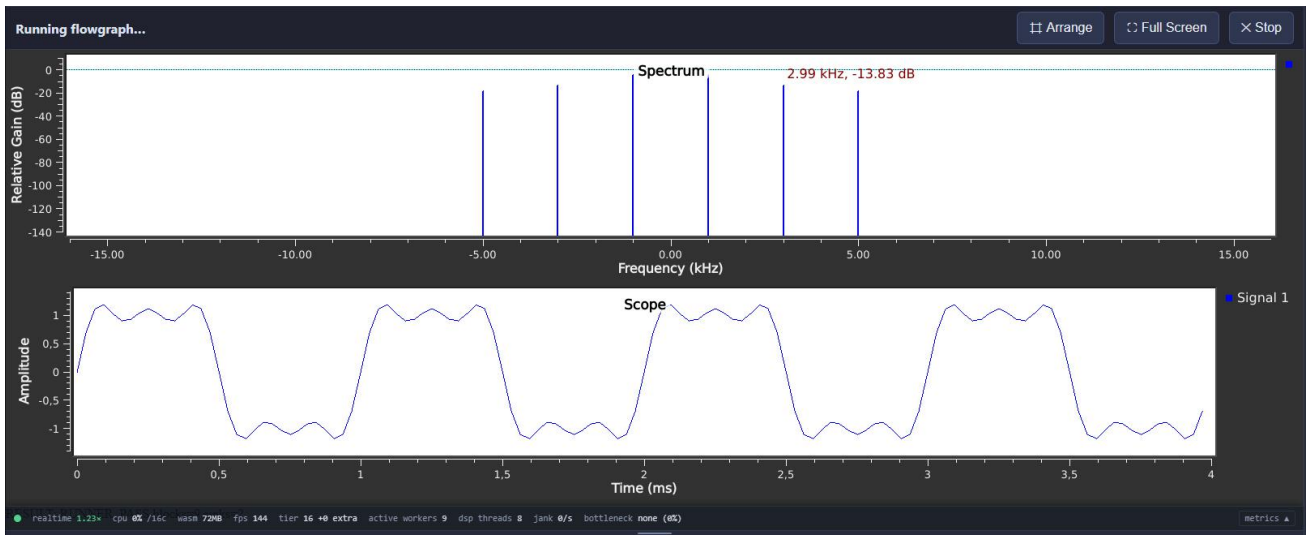
- Đo trên Frequency Sink độ chênh lệch theo dB giữa vạch 1 kHz và vạch 3 kHz.

Frequency Sink tại:

- 1 kHz: **-4.02 dB**

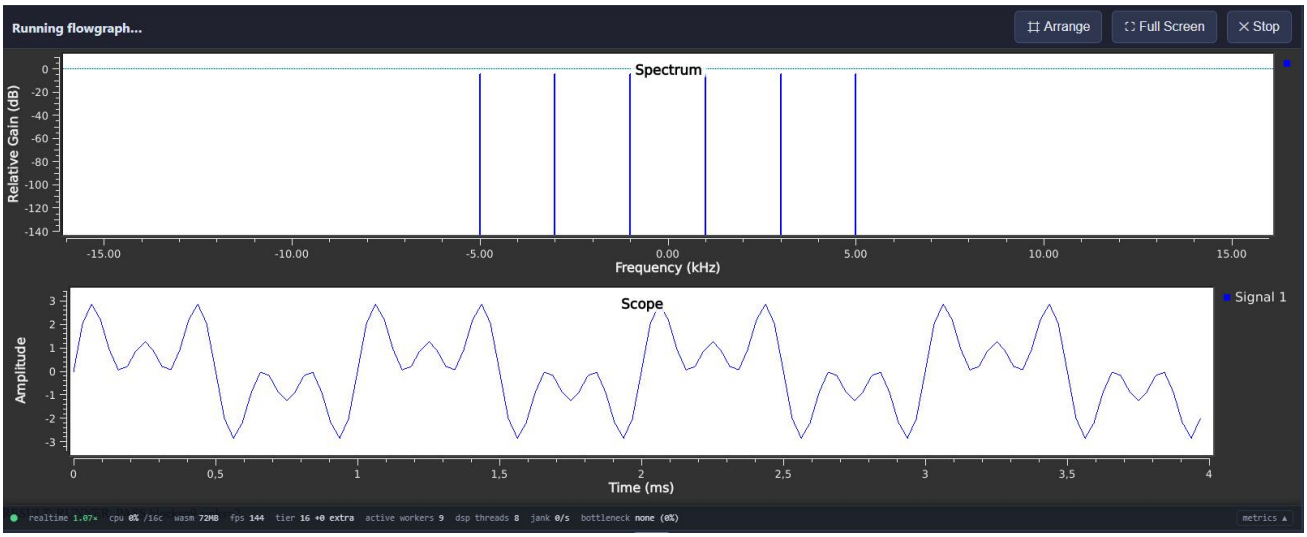


- 3 kHz: **-9.63 dB**



  - Đối chiếu với giá trị lý thuyết $20 \cdot \log_{10}(3)$:

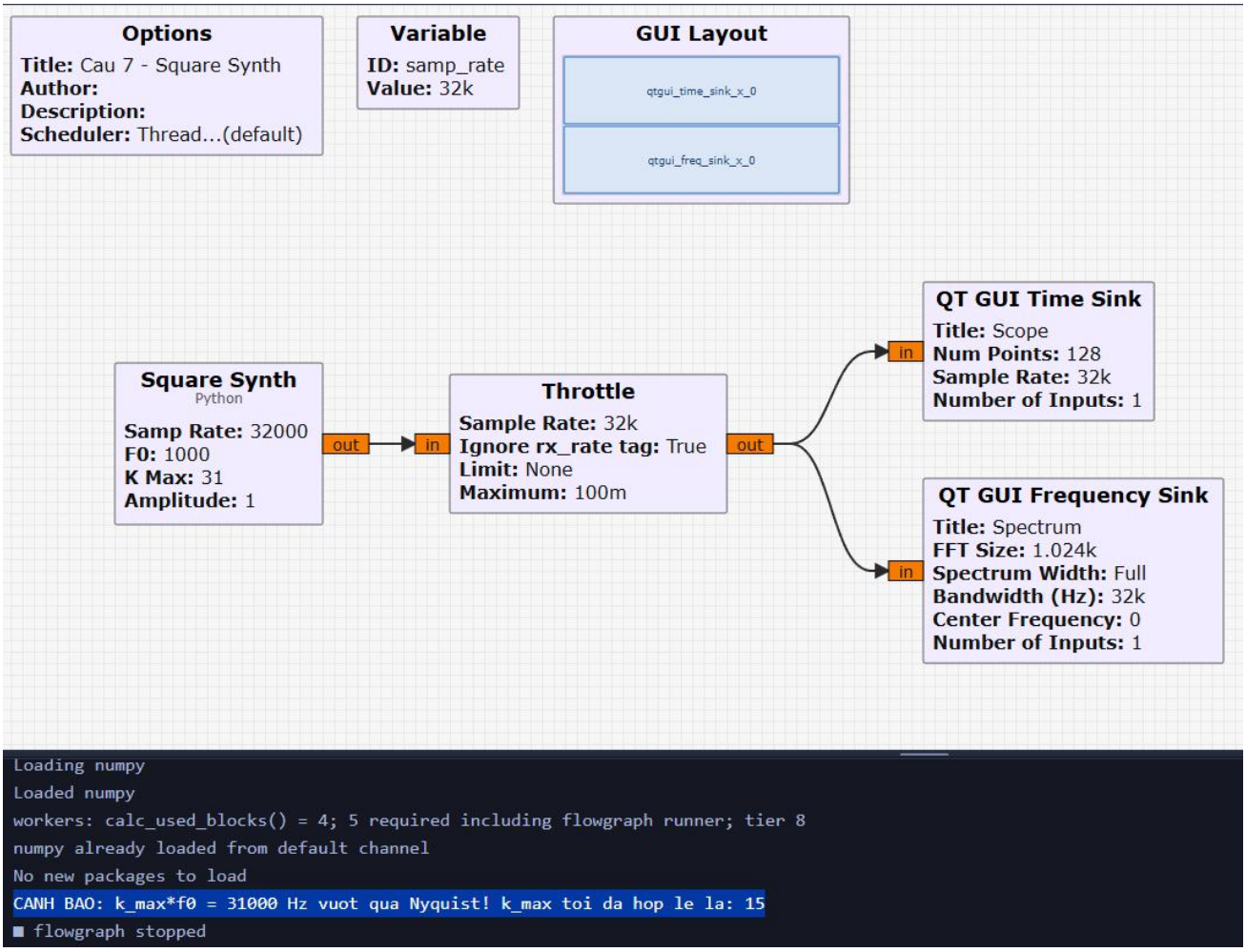
- Độ chênh lệch chính là khoảng cách giữa hai đỉnh vạch phổ:
  $\Delta = |(-4.02) - (-13.83)| =$ **9.81** dB
- **Kết luận:** Kết quả đo được (9.81 dB) **rất sát** với lý thuyết (9.54 dB). Sai số khoảng 0.27 dB là hoàn toàn chấp nhận được trong mô phỏng
  - Thử cố tình đặt sai biên độ của cả ba nguồn thành 1,0 (không chia cho k). Chạy lại và giải thích vì sao dạng sóng không còn giống sóng vuông.



- **Nhận xét:** Khi thay đổi biên độ của cả 3 khối Signal Source thành 1.0, dạng sóng trên đồ thị Scope bị méo mó nghiêm trọng, mất đi hình dáng sóng vuông và xuất hiện các gai nhọn. Trên đồ thị Spectrum, độ cao của 3 vạch phổ (1kHz, 3kHz, 5kHz) gần như bằng nhau.
- **Giải thích:** Theo lý thuyết chuỗi Fourier, sóng vuông được tổng hợp từ các sóng hài bậc lẻ, trong đó biên độ của các hài phải giảm dần theo tỉ lệ 1/k. Việc đặt biên độ cả 3 bằng 1.0 đã phá vỡ quy luật này, làm cho năng lượng của các hài bậc cao (3kHz, 5kHz) trở nên quá lớn so với thực tế. Sự mất cân bằng năng lượng này khiến tín hiệu tổng hợp không thể tạo ra dạng sóng vuông phẳng mà bị biến dạng thành các gợn sóng và gai nhọn.

8) **Viết khối nguồn Square Synth** để mô phỏng quá trình tổng hợp sóng vuông. Thay vì sử dụng 3 khối Input như Hình 2.8, yêu cầu viết **một khối nguồn** tự sinh ra tổng các sóng, với số **k** là một tham số chỉnh được ngay trên giao diện. (Sử dụng khối Embedded Python Block đã học ở Bài 1, nhưng lần này là khối **nguồn**: không có cổng vào)

   - Thêm tham số `amplitude` (mặc định 1,0) nhân vào toàn bộ đầu ra.
   - Thêm phần kiểm tra Nyquist ngay trong `__init__`: nếu `k_max * f0 >= samp_rate / 2` thì in ra cảnh báo kèm giá trị `k_max` lớn nhất còn hợp lệ.

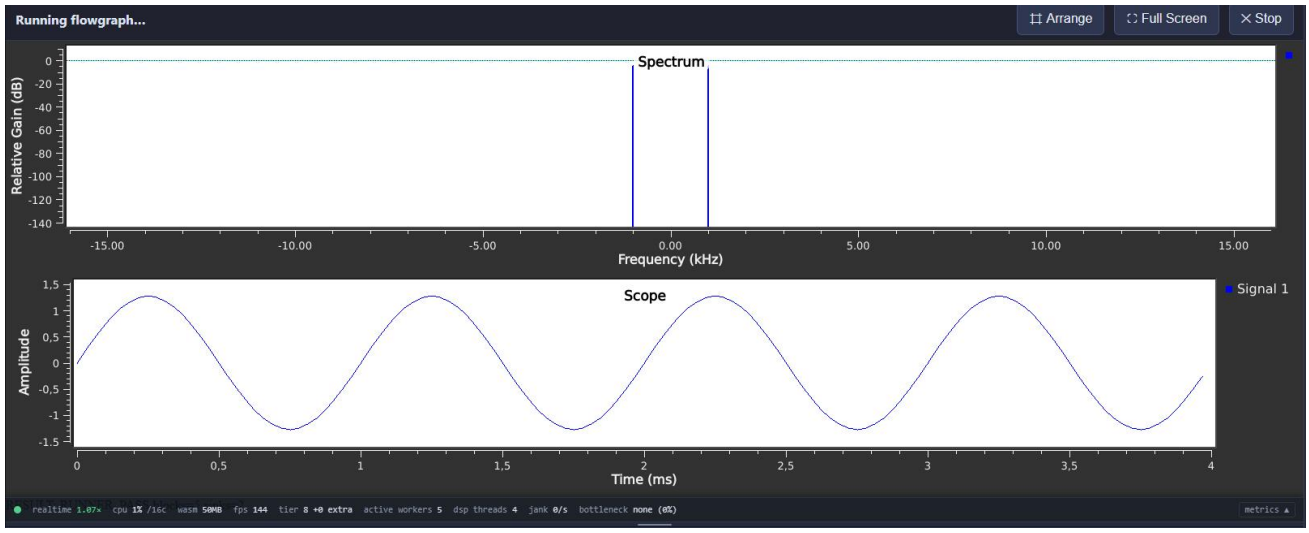


Giải thích: Khi chạy flowgraph với k_max = 31 và f0 = 1000 Hz, tần số của sóng hài cao nhất là 31 kHz. Do tần số lấy mẫu là 32 kHz, tần số Nyquist là 16 kHz. Việc 31 kHz > 16 kHz đã vi phạm định lý lấy mẫu Nyquist, gây ra hiện tượng chồng phổ (Aliasing). Khối Square Synth đã tự động phát hiện và in ra cảnh báo trên console: k_max*f0 = 31000 Hz vuot qua Nyquist! k_max toi da hop le la: 15. Trên đồ thị Spectrum, các hài bậc cao bị gập ngược trở lại tần số thấp, làm sai lệch phổ. Trên đồ thị Scope, dạng sóng không còn giống sóng vuông mà bị biến dạng, gợn sóng mạnh do sự giao thoa của các tần số bị gập.

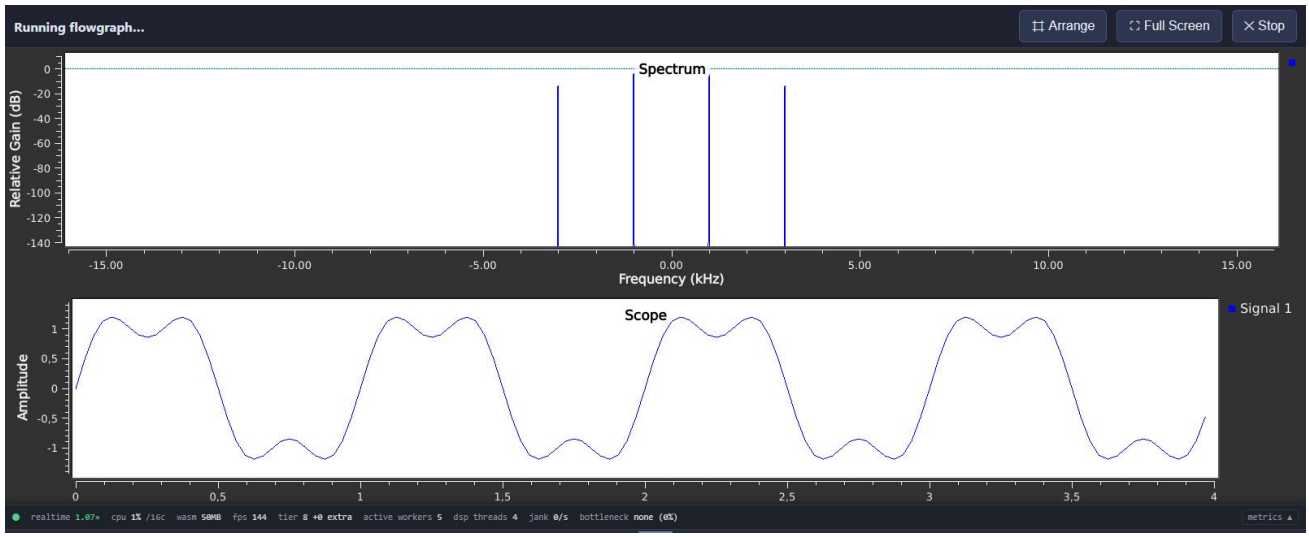
- Gắn vào flowgraph như ví dụ bên dưới, chạy với `k_max` = 1, 3, 11, 31 và chụp lại dạng sóng cùng phổ.

Các trường hợp:

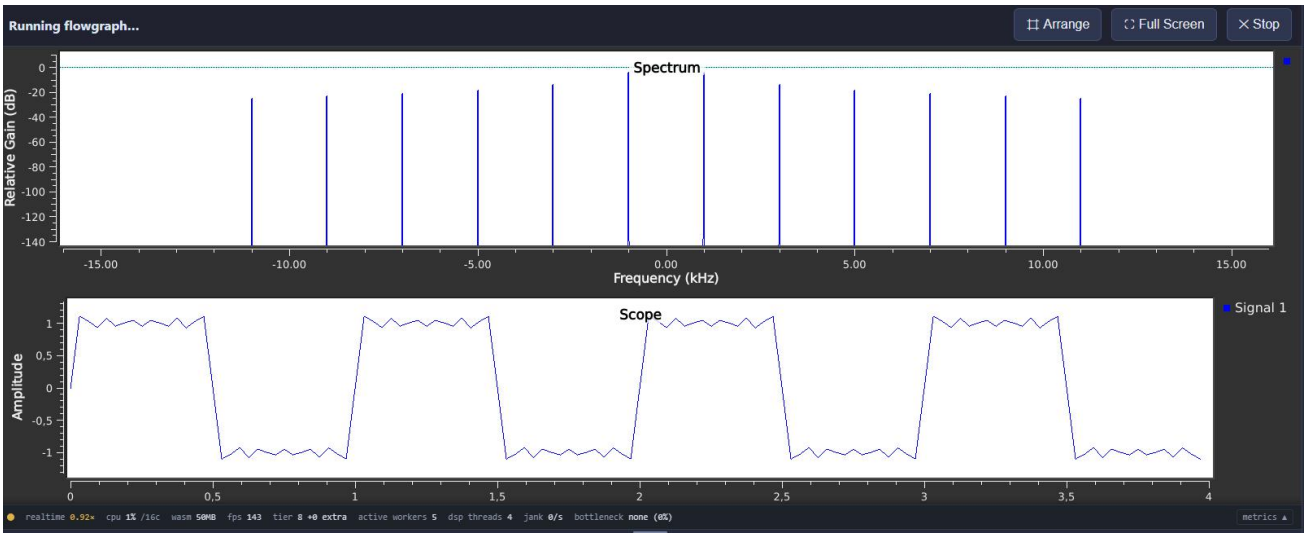
- k_max = 1:



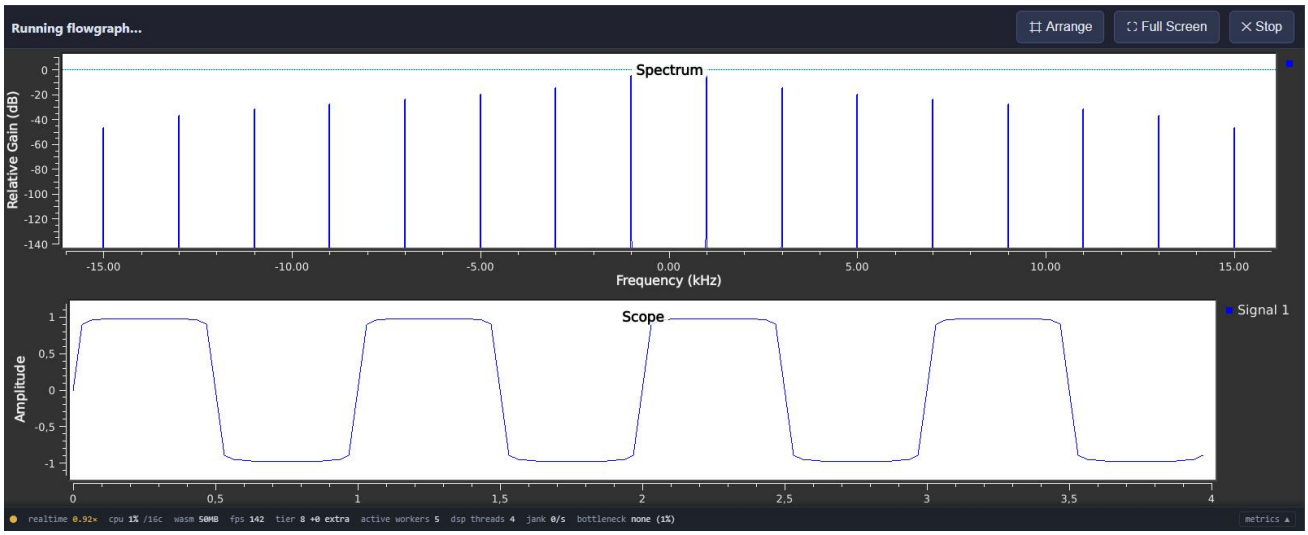
- k_max = 3:



- k_max = 11:



- k_max = 31:



- So sánh công sức dựng sơ đồ giữa cách này và cách ở **Bài 9** khi cần k = 31.

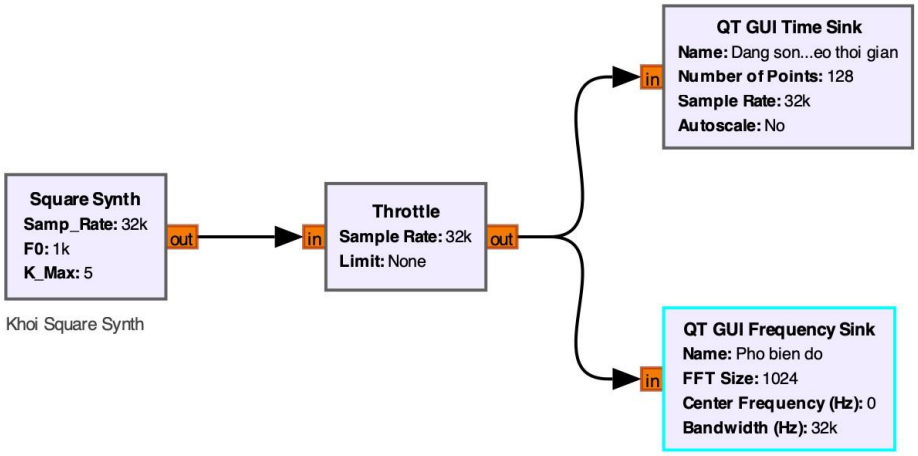
Nhận thấy đề bài có thể có sự nhầm lẫn nhỏ giữa Câu 7 và Câu 9, em xin đưa ra so sánh như sau:

- **Nếu so sánh với cách dùng khối có sẵn (giống Câu 7):** Để tổng hợp sóng với $k = 31$bằng các khối Signal Source, ta phải tạo ra tới 16 khối (cho các hài bậc lẻ), nhập thủ công tần số và biên độ cho từng khối, sau đó nối dây vào khối Add cực kỳ cồng kềnh và dễ sai sót. Trong khi đó, ở Câu 8, chỉ cần dùng **1 khối Square Synth** tự viết và thay đổi tham số K Max = 31 là xong. Rõ ràng cách viết khối Python ở Câu 8 tiết kiệm công sức hơn rất nhiều.
- **Nếu so sánh với Bài 9 (Khối Channel):** Cả hai bài đều sử dụng khối Embedded Python Block. Tuy nhiên, ở Bài 9, việc dựng sơ đồ rất đơn giản vì khối Channel chỉ

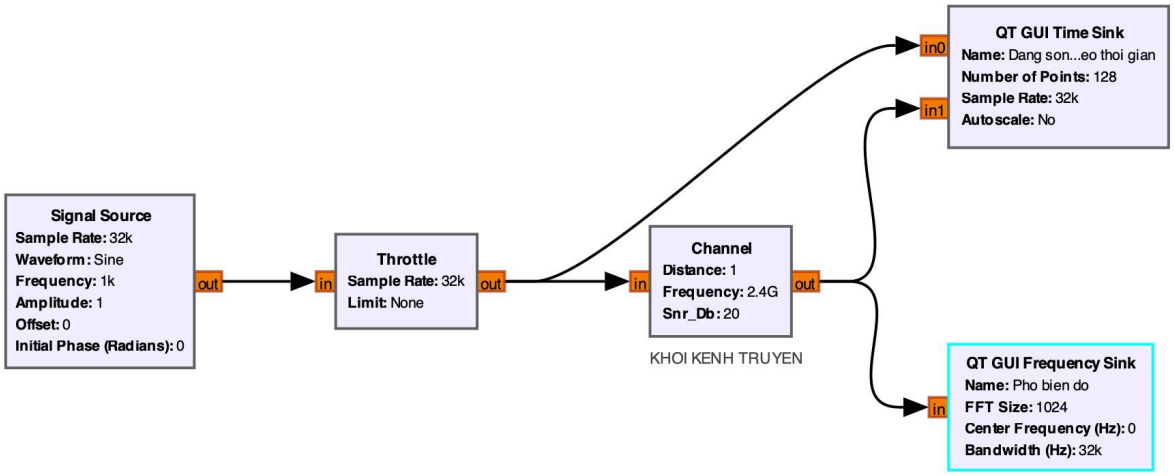
có 1 input, 1 output và các tham số cố định (khoảng cách, tần số, SNR). Còn ở Câu 8, khối Square Synth phức tạp hơn vì phải tự tính toán vòng lặp chuỗi Fourier, quản lý pha và đặc biệt là phải xử lý bài toán kiểm tra điều kiện Nyquist bên trong code. Do đó, viết code cho Câu 8 đòi hỏi công sức và tư duy thuật toán nhiều hơn Bài 9.

Gợi ý:

- Khối nguồn khai báo in_sig=None. Khi đó input_items là danh sách rỗng, và số mẫu cần sinh ra lấy từ len(output_items[0]).



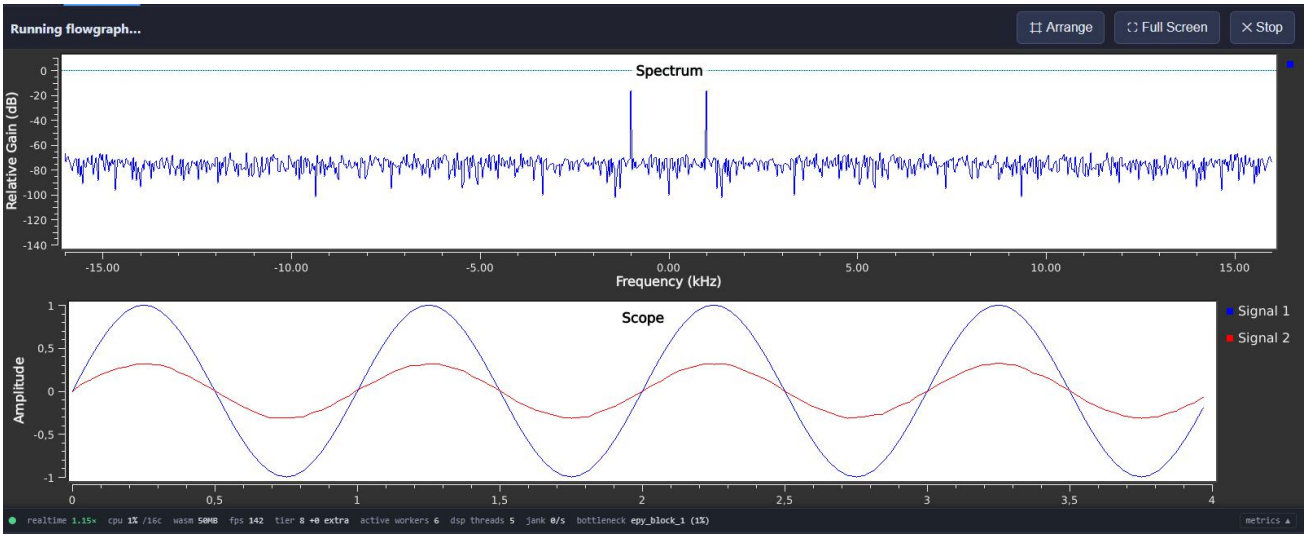
9) **Mô phỏng kênh truyền trên GNU Radio.** Để đưa suy hao vào mô phỏng, hãy viết một khối xử lý *nhận tín hiệu vào*, nhân với *hệ số suy hao* tính từ khoảng cách, rồi cộng thêm *nhiễu*. Sinh viên thiết kế khối Channel và dựng flowgraph giống hình bên dưới.



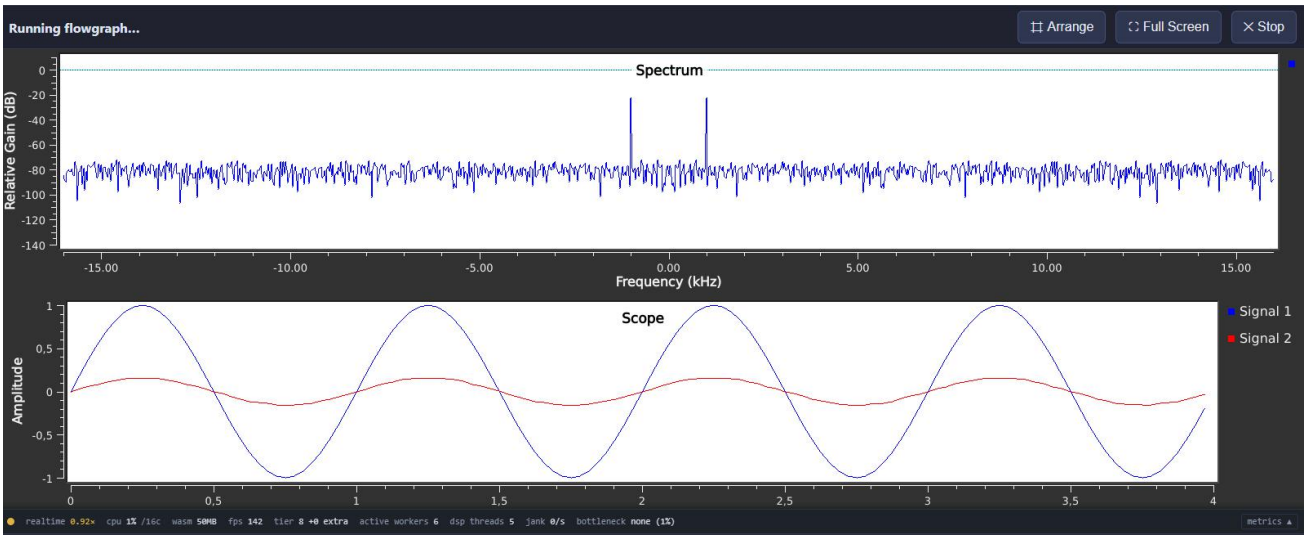
- Dựng flowgraph như trên, đặt *Number of Inputs* của Time Sink bằng 2 để xem đồng thời tín hiệu gốc và tín hiệu sau kênh.
- Giữ `snr_db` = 30, lần lượt đặt `distance` bằng 1, 2, 4 và 8 m. Với mỗi giá trị, đọc biên độ tín hiệu ra trên Time Sink và lập bảng.

Các trường hợp:

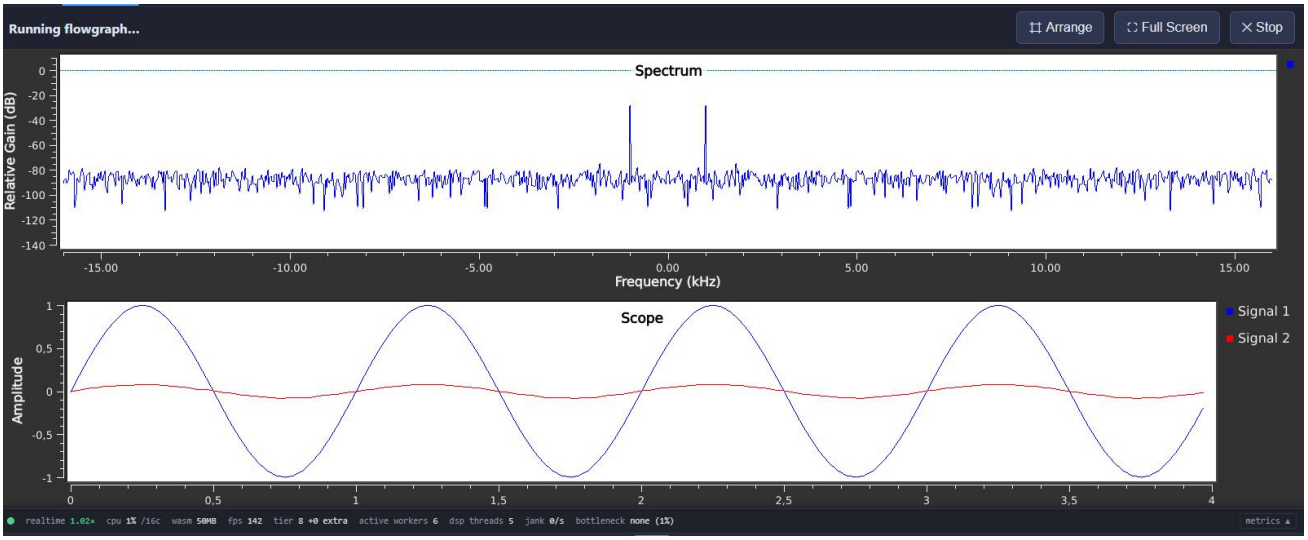
- Distance = 1:



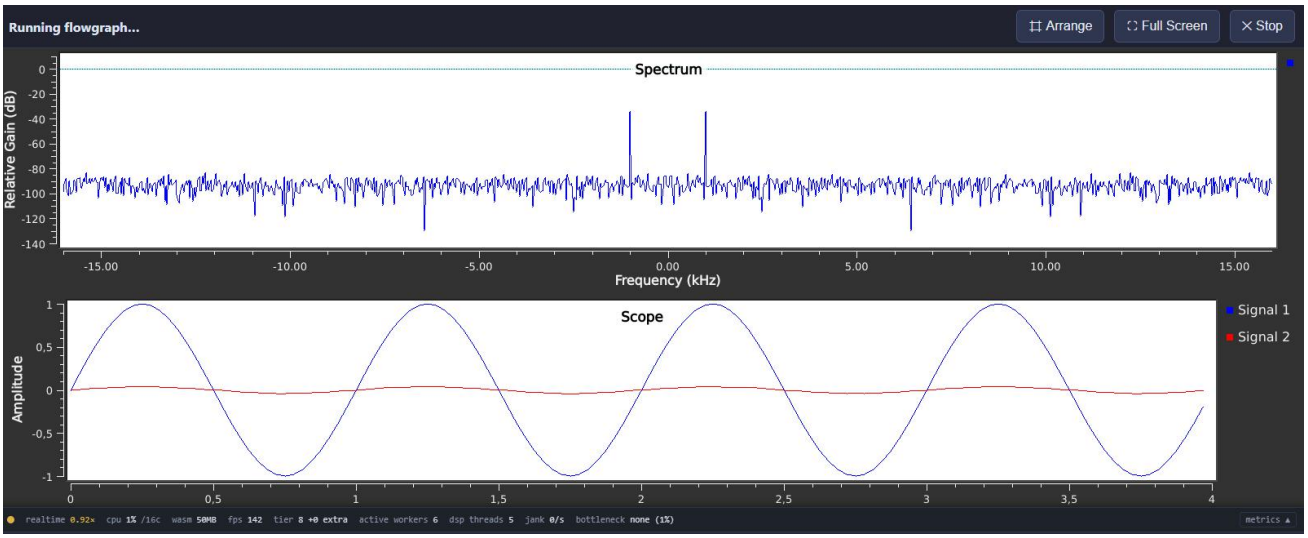
- Distance = 2:



- Distance = 4:



- Distance = 8:



- Tính tỉ số biên độ giữa hai lần đo liên tiếp.

Khoảng cách (d)	Biên độ đỉnh đo được (Signal 2)	Tỉ số biên độ giữa 2 lần đo liên tiếp
1 m	0.32	-
2 m	0.16	0.32 / 0.16 = 2.0
4 m	0.08	0.16 / 0.08 = 2.0
8 m	0.04	0.08 / 0.04 = 2.0

Yêu cầu nộp bài

- Một file báo cáo theo mẫu (định dạng PDF) trình bày lời giải, hình vẽ kết quả và phần nhận xét, giải thích cho từng câu.
- Mã nguồn Python (`.py`) đặt tên theo định dạng `MSSV_HoTen_Lab02_Bai1.py`, có chú thích rõ ràng cho từng phần.
- Các file flowgraph `.grc` và file mã nguồn của các khối Python custom.
- Ảnh chụp màn hình cửa sổ Time Sink và Frequency Sink của các flowgraph đã dựng.

Lưu ý
Sai hình thức báo cáo: trừ 2 điểm (gồm nộp đúng định dạng pdf và làm báo cáo theo mẫu).

Ảnh chụp màn hình chỉ có giá trị khi đi kèm phần **giải thích và nhận xét**. Báo cáo chỉ có ảnh mà không có lý giải sẽ không được tính điểm phần đó. Giải thích đơn giản, đúng trọng tâm.

Không chia sẻ tài liệu và mã nguồn ra ngoài lớp học khi chưa có sự đồng ý của giảng viên.